Lecture 2: Supervised Learning, Linear Regression, maybe Regularized Regression

Max G'Sell

Machine Learning I: 46-926

November 2, 2020

Announcements

- Current office hour schedule:
  - Monday, 10:20am: Me
  - Tuesday, 5pm: Vaibhav ← or 9am?
  - Tuesday, 8pm: Wanshan

  How does this look? Feedback welcome.

- First homework is due Wednesday *before class*.
  - Comment about gradescope

Submit one PDF, readable

Today: Finishing basics, Linear Regression and Regularization

- Finishing up basic supervised learning, bias/variance tradeoff
- Returning to linear regression
- Extending linear regression to work in high dimensions
  - Ridge Regression
  - Lasso

Review: Supervised learning setup

We are going to focus on prediction $(Y \in \mathbb{R})$ for a little while.

We observe independent draws (x, y) from a fixed distribution $\mathcal{P}$, with $x \in \mathbb{R}^p$. We observe a training set of n examples,

$(x_1, y_1), \ldots, (x_n, y_n)$. From these, we want to build a function $\hat{\mu}(x)$ that takes vectors $X \in \mathbb{R}^p$ to guesses at Y.

We measure the quality of our predictions with a *Loss function* $\mathcal{L}(Y, \hat{\mu}(X))$, which is larger for less favorable guesses.

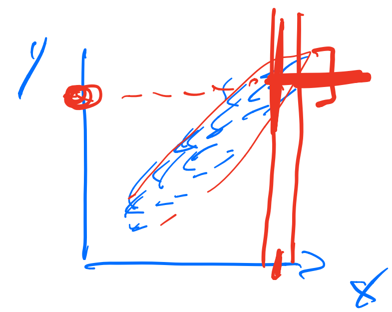
We try to construct $\hat{\mu}$ to minimize the *expected loss*. For example:

$$\mathbb{E}\mathcal{L}(Y, \hat{\mu}(X)) = \mathbb{E} \left(Y - \hat{\mu}(x)\right)^2$$

Review: Regression function

We saw two important results about minimizing squared error loss last class.

The Regression Function. If we were given the generating distribution $\mathcal{P}$ that (X, Y) pairs are drawn from, then the best possible function we could choose is



$$\mu(x) = E[Y | X = x]$$

The best we can do, once we have seen x, is to look at the conditional distribution of Y given that $X = x$, and take the mean.

Again, this is only true for squared-error loss! (Homework 1 Problem 2)

Review: Bias-variance decomposition

If we don't know $\mathcal{P}$, then we have to use our training set to estimate our function $\hat{\mu}$.

In what ways can we be wrong?

$$\mathbb{E}\left[\mathcal{L}(Y, \hat{\mu}(X)) \mid X = x\right] = \mathbb{E}\left[(Y - \hat{\mu}(X))^2 \mid X = x\right]$$

$$= \sigma_x^2 + \left(\mathbb{E}\left[\hat{\mu}(X) | X = x\right] - \mu(x)\right)^2$$

$$+ \mathbb{E}\left[\left(\hat{\mu}(X) - \mathbb{E}\left[\hat{\mu}(X) | X = x\right]\right)^2 \mid X = x\right]$$

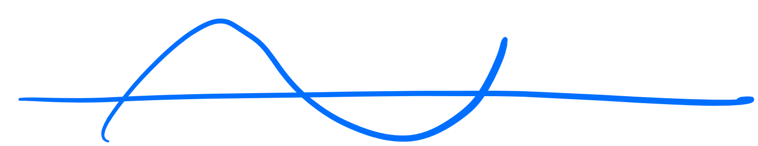
$$= \sigma_x^2 + \operatorname{bias}\left(\hat{\mu}(x)\right)^2 + \operatorname{var}\left(\hat{\mu}(x)\right)$$

Bias-variance decomposition

$$\mathbb{E}\left((Y-\hat{\mu}(X))^2 \,|X=x\right)=\sigma_x^2+\text{bias}\left(\hat{\mu}(x)\right)^2+\text{var}\left(\hat{\mu}(x)\right)$$

This gives you a way of thinking about where errors in predictions come from.

If your estimator is too inflexible to approximate the true mean function, it is too Biased.



If your estimator
adapts too much to individual training samples and over fits, it is too

Variable.

We will see many examples.

Approximating the regression function

We know that the best we could do is use $\mu(x) = \mathbb{E}(Y|X = x)$ as our predictor, but we don't have access to it.

Instead, we have a sample $(x_1, y_1), \ldots, (x_n, y_n)$ of examples. We want to use these examples to build an estimate $\hat{\mu}(x)$.

We will see many ways of constructing such an estimate. We'll begin with the simplest: Linear Regression.

Example: Linear regression

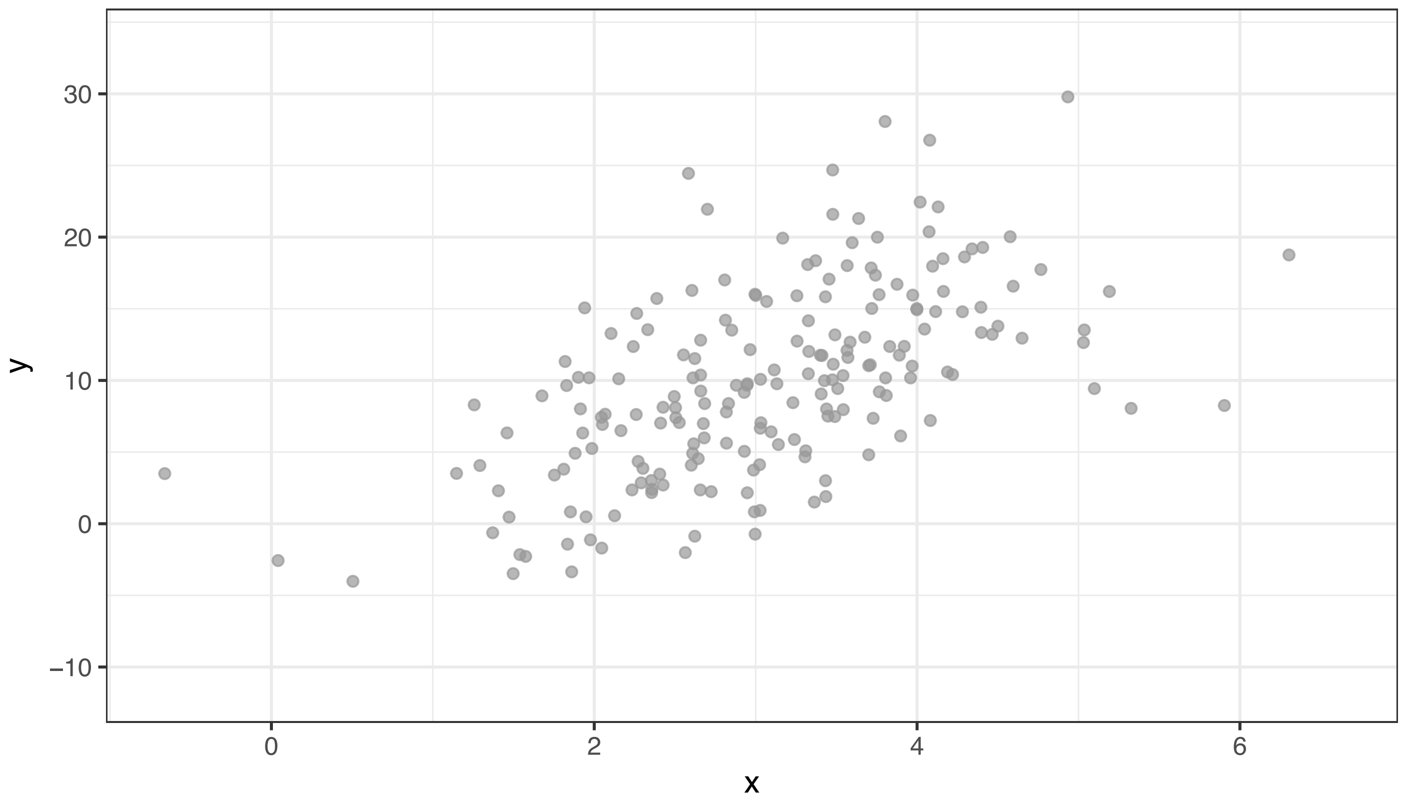
For machine learning purposes, we think of regression as a very simple way to approximate $\mathbb{E}(Y|X)$. While $\mathbb{E}(Y|X)$ could be any function, we decide to approximate it as a linear function of the predictors.

$$\begin{aligned}\hat{\mu}(x) &= \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p \\ &= x^T \hat{\beta}\end{aligned}$$

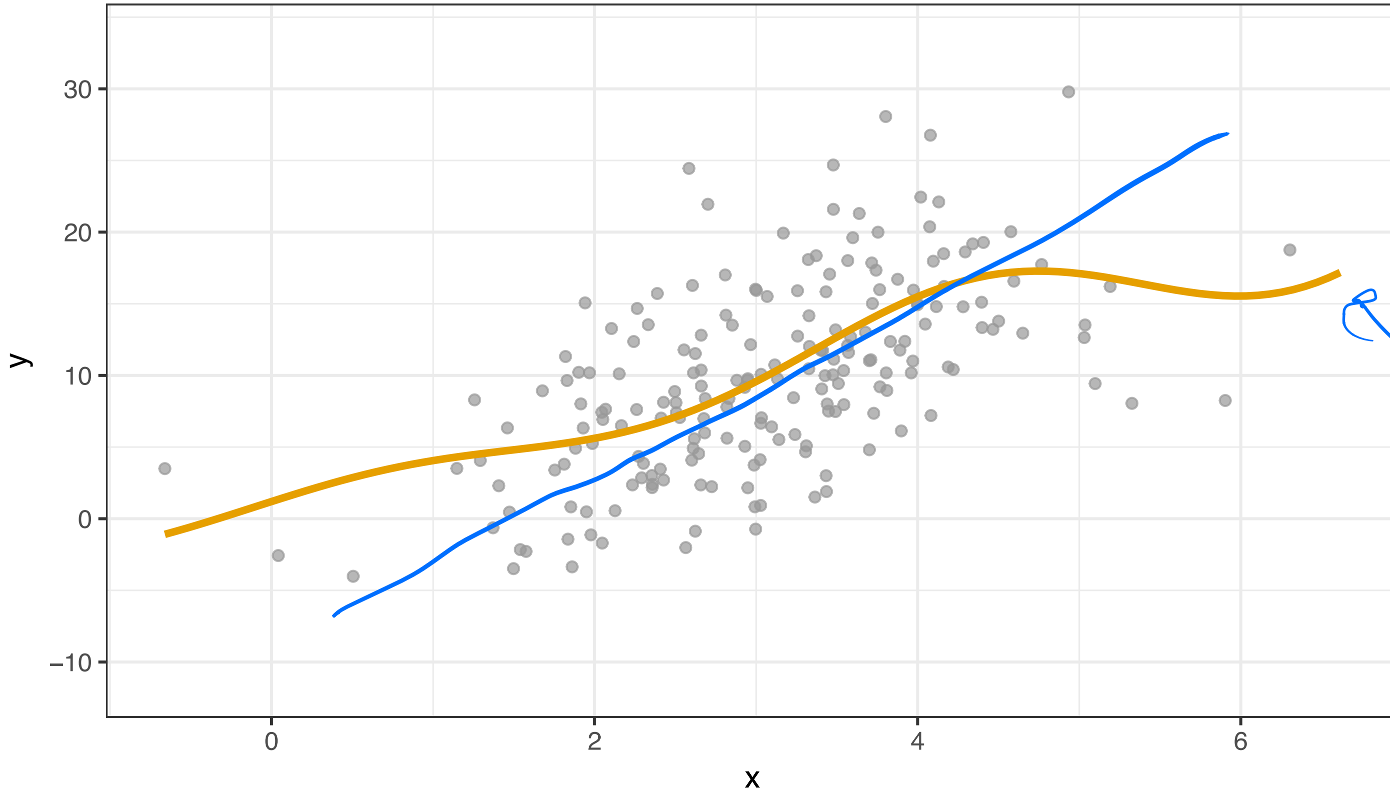
We usually estimate $\hat{\beta}$ using least squares, as you learned. Assuming a linear model simplifies things tremendously, but may also cause its own problems.

Our view of regression for prediction is quite a bit different from the common statistical view of regression. *We don't need to believe that the linear model assumptions are at all true!* We just hope it makes good guesses.

Regression function demo

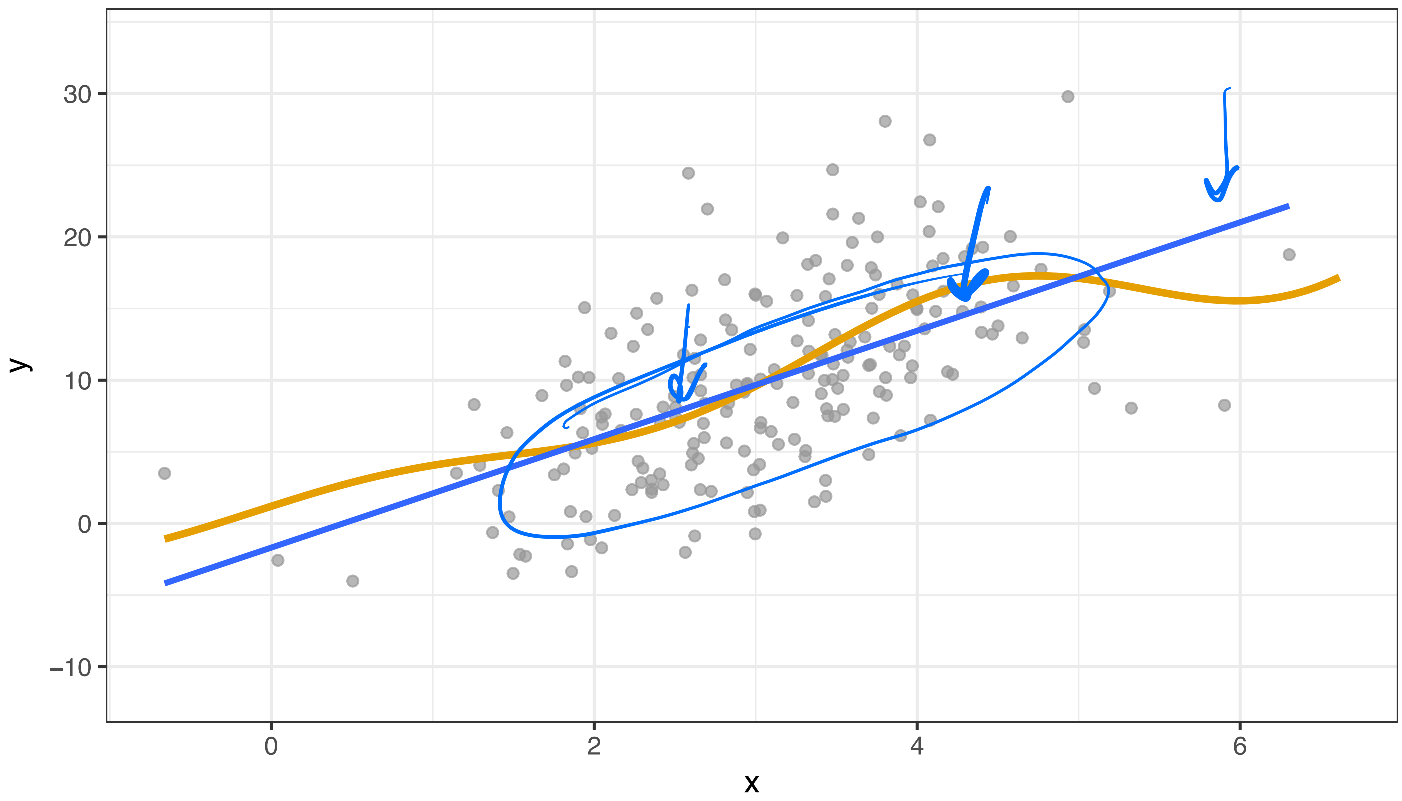


Regression function demo

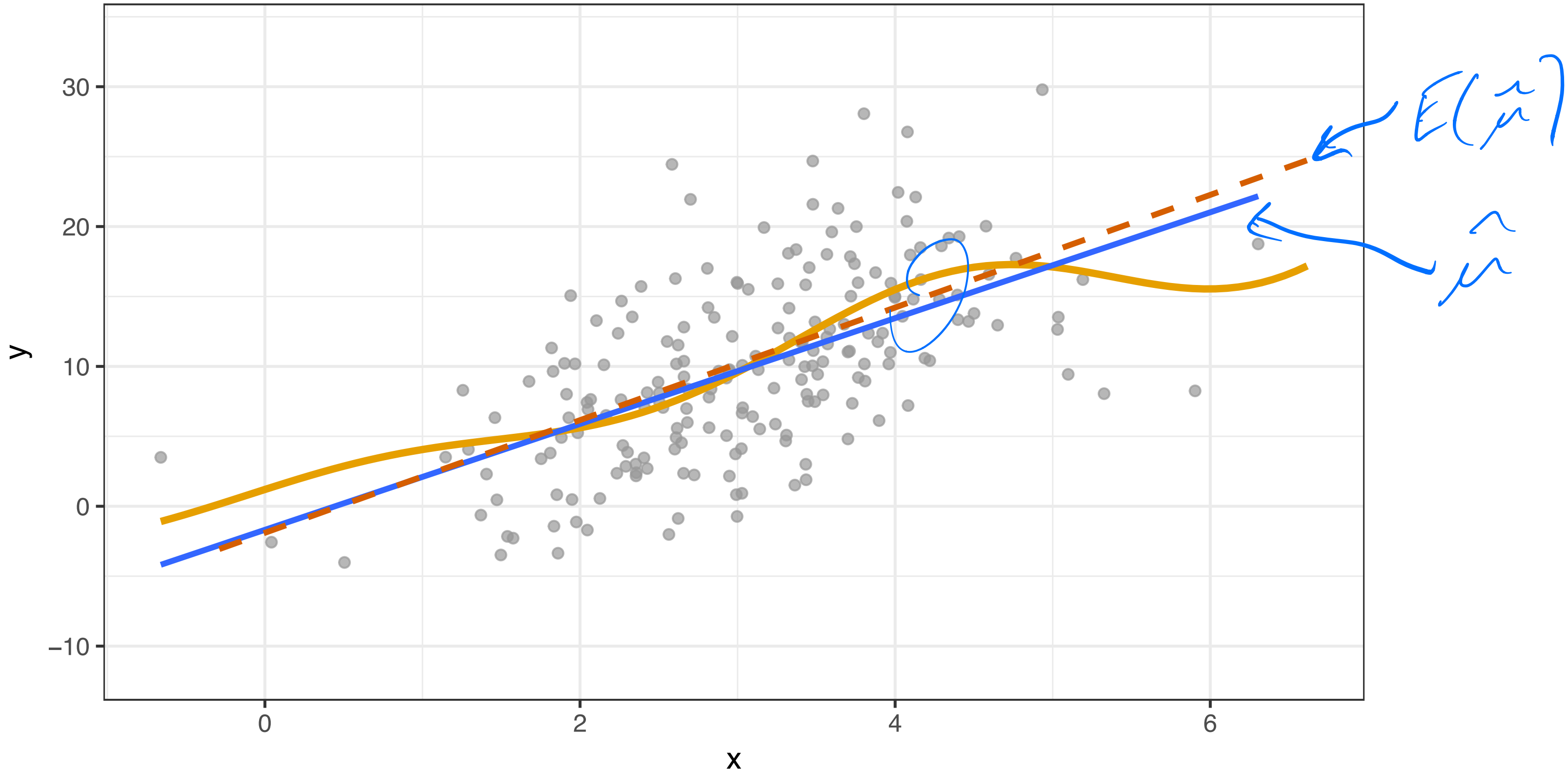


$E(Y|X)$

Regression function demo



Regression function demo



Linear regression

$$\begin{aligned}\hat{\mu}(x) &= \hat{\beta}_0 + \hat{\beta}_1 x_1 + \hat{\beta}_2 x_2 + \cdots + \hat{\beta}_p x_p \\ &= x^T \hat{\beta}\end{aligned}$$

If the true $\mathbb{E}(Y|X)$ is very far from linear, this model will be highly *biased*, since even $\mathbb{E}\hat{\mu}(X)$ will never be able to match $\mathbb{E}(Y|X)$.

Even with infinite data, the best linear model would still be terrible. You learned (and we will discuss) much more flexible models to avoid this problem.

We will have other problems even when the linear model is correct!

Review: Linear Regression

You've already seen a statistical perspective on linear regression last Mini in Data Science.

Suppose that we observe n copies of $(X_1, \ldots, X_p, Y)$. We can fit the model

$$Y = \beta_0 + \beta_1 X_1 + \ldots \beta_p X_p + \varepsilon,$$

i^{th} obs

X_{ij} j^{th} var

by minimizing the RSS:

$$RSS(\beta) = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$
$$= \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2$$

(Note: I'm not going to bother with the intercept after this to keep notation pleasant, you can just pretend that $X_1 = 1$.)

Review: regression with multiple predictors

We will almost always use the matrix formulation of linear regression. The same notation will be useful in other topics.

Suppose that we observe n copies of $(X_1, \ldots, X_p, Y)$. We can stack all the X values into a matrix $\mathbf{X}$,

$$\mathbf{X} = \overbrace{\begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}}^{p} \left.\vphantom{\begin{pmatrix} x \\ x \\ \vdots \\ x \end{pmatrix}}\right\} n$$

Note that the rows $x_i \in \mathbb{R}^p$, $x_i = (x_{i1}, \ldots, x_{ip})$, are the individual observations, and the columns $\mathbf{x}_j \in \mathbb{R}^n$, $\mathbf{x}_j = (x_{1j}, \ldots, x_{nj})^T$, are all the observations of a particular variable.

We can then write the corresponding linear model for all the Y very simply as $Y = X\beta$, or expanded,

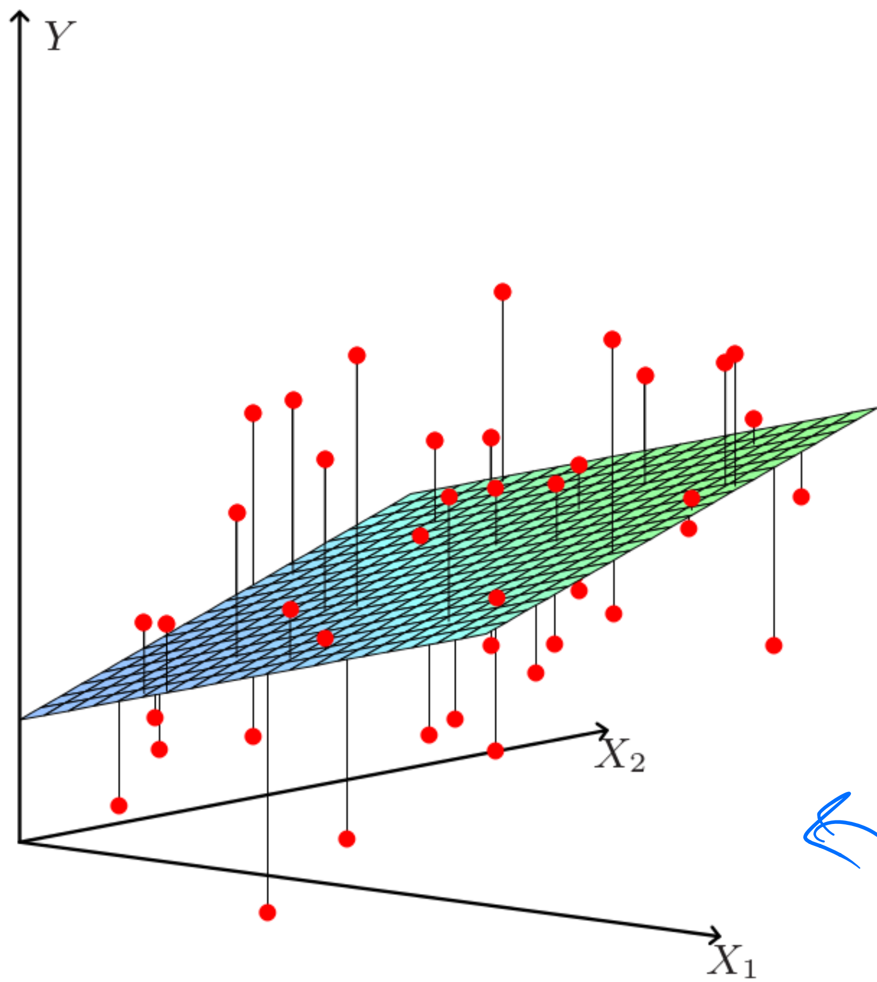
$X\beta$

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix} \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_p \end{pmatrix}$$

Why do we care? The linear algebra reformulation of the problem gives us geometric intuition into regression, which will carry over into more complicated ML procedures.

(It will also make derivations of classic statistical results much easier, though we won't get into that here.)

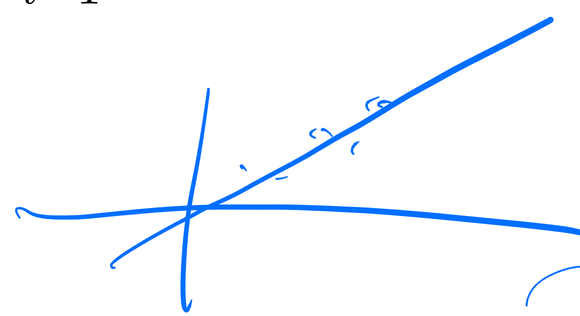
Two Pictures: p-space



$$n\left\{\overbrace{\begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}}^{p+1}\right.$$

Views the observations
$\underline{(x_{i1}, \ldots, x_{ip}, y_i)}$ as points in $\mathbb{R}^p$

$$y_i = \sum_{i=1}^{p} x_{ij}\beta_j = x_i^T\beta$$



x

Easy to see:

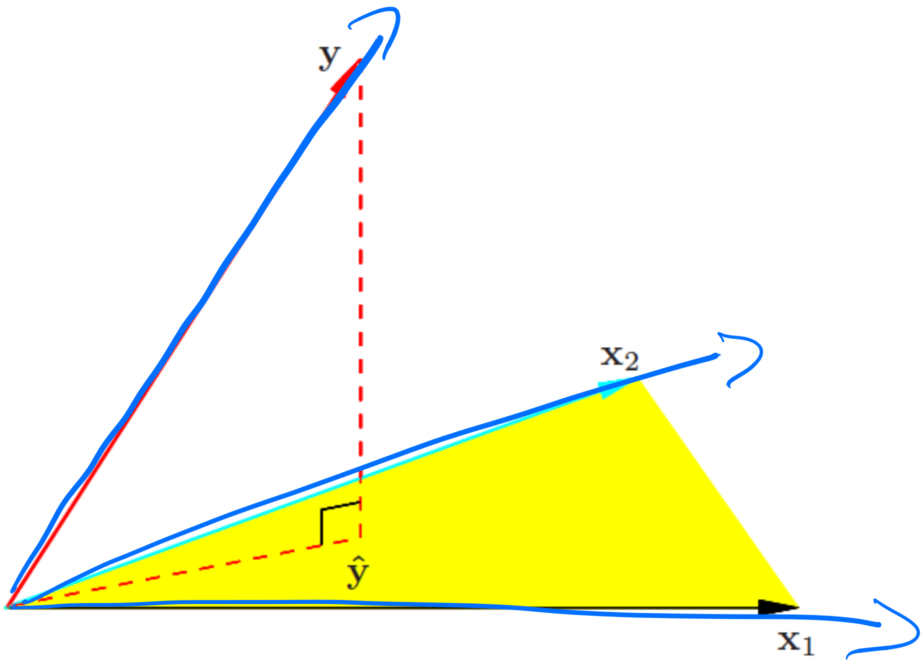
- What a new observation does
- How well individual y_i are approximated

Two Pictures: n-space

p+1 vectors

$$\begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, \quad \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}$$

$\in \mathbb{R}^n$ $\quad\quad$ $\in \mathbb{R}^n$



Views the columns $(x_{1j}, \ldots, x_{nj})$ as vectors in $\mathbb{R}^n$, along with $(y_1, \ldots, y_n)$ and $(\varepsilon_1, \ldots, \varepsilon_n)$

$$\hat{Y} = \sum_{i=1}^{n} X_j \beta_j$$

$$Y = \begin{pmatrix} x_{11} \\ x_{21} \\ x_{31} \end{pmatrix} \beta_1 + \begin{pmatrix} x_{12} \\ x_{22} \\ x_{32} \end{pmatrix} \hat{\beta}_2$$

Easy to see:

- How to fit our model
- What happens when new variables are added?
- How the error ε interacts with dimension?

$v \in \mathbb{R}^n$

Minimizing the RSS now means minimizing

$\|v\|_p = \left(\sum_{i=1}^{n} |v_i|^p\right)^{1/p}$

$\|v\|_2 = \sqrt{\sum v_i^2}$

$\|v\|_1 = \sum |v_i|$

$\|v\|_\infty = \max_i |v_i|$

$$RSS(\beta) = \sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{p} x_{ij}\beta_j \right)^2$$

$$= \sum_{i=1}^{n} (Y - X\beta)_i^2$$

$$=$$

$$= \|Y - X\beta\|_2^2$$

$$Y = \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

$$\begin{pmatrix} x_1^T\beta \\ x_2^T\beta \\ x_3^T\beta \end{pmatrix} = X\beta$$

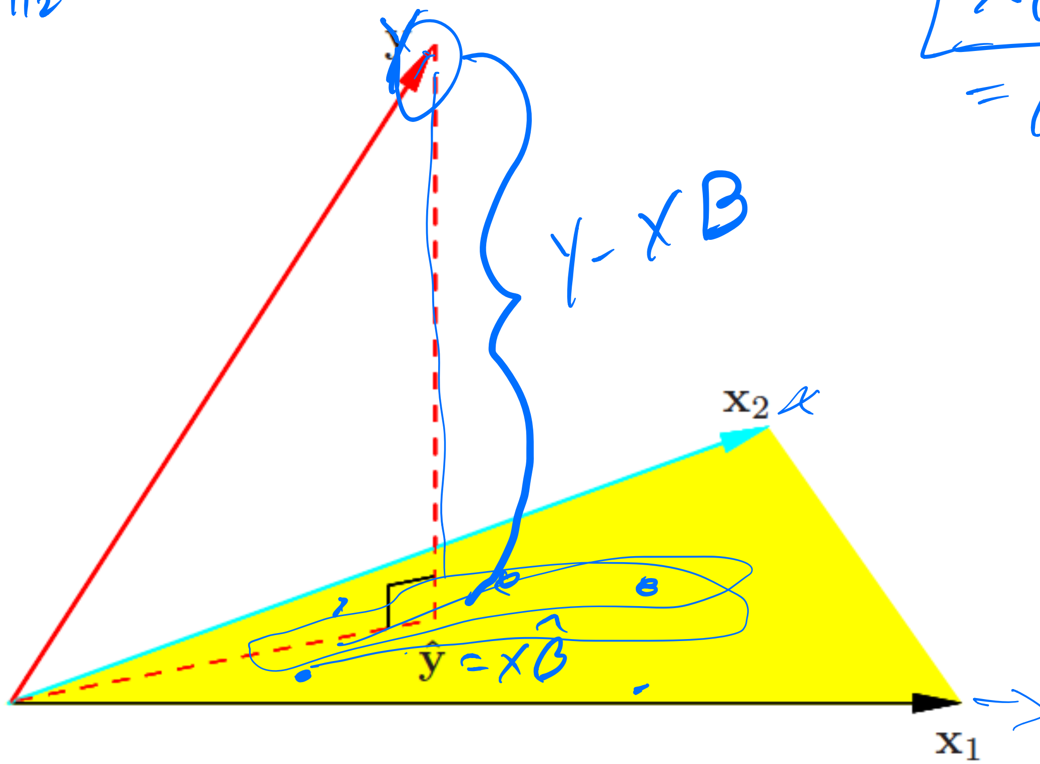
Let's get an intuition for how $\hat{Y} = X\hat{\beta}$ behaves.

We can write $\hat{Y} = X\hat{\beta} = \sum_{j=1}^{p} \hat{\beta}_j X_j$. The last term is clearly a linear combination of the columns of X, so we know that $\hat{Y}$ lies in the column span of X (a linear subspace).

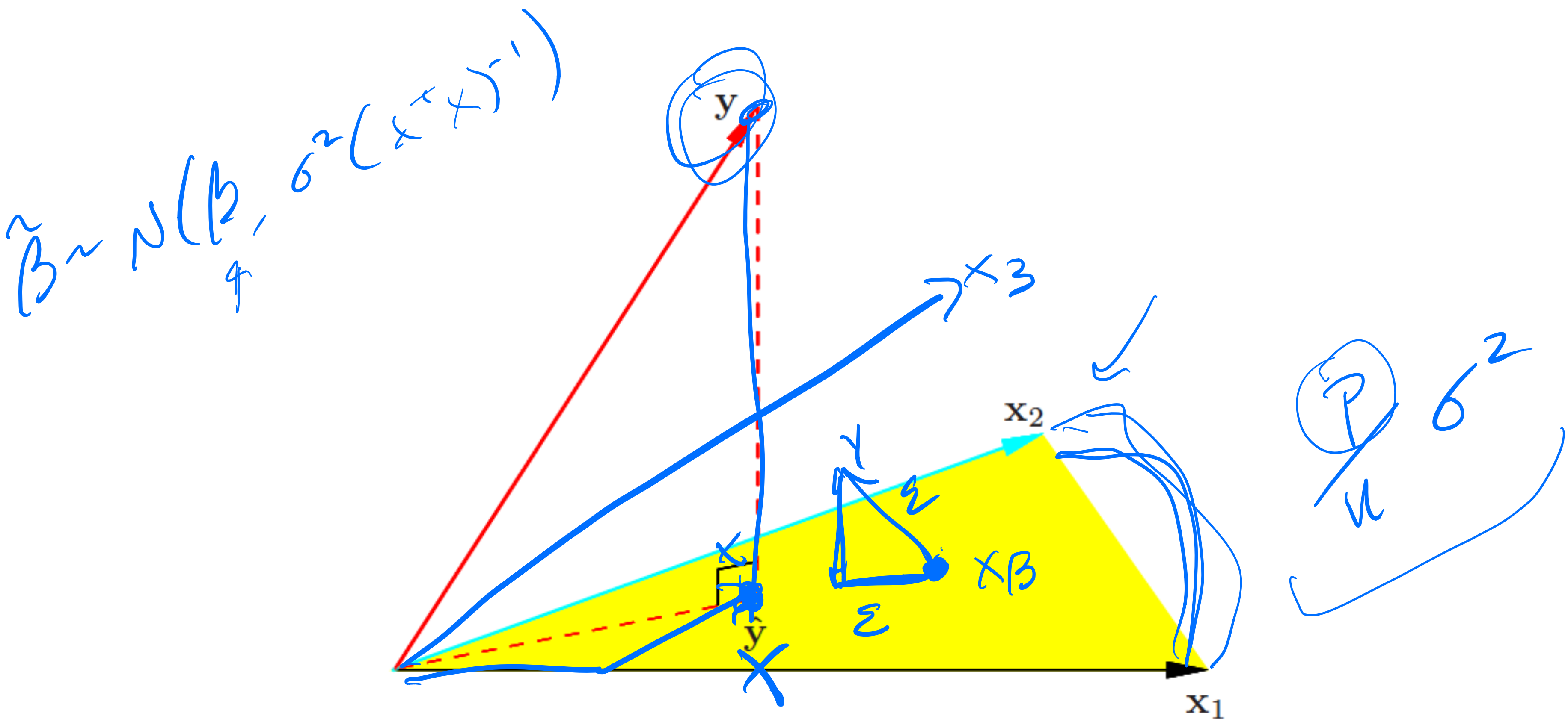
$\boxed{X\beta} \in \text{span}(x_1, x_2 \ldots)$

$= \beta_1 X_1 + \beta_2 X_2 + \ldots$

$RSS = \|Y - X\beta\|_2^2$



(ESL pg. 46)



$\hat{Y}$ minimizes $\|Y - \hat{Y}\|_2^2$, the euclidian distance from Y to $\hat{Y}$.

Since $\hat{Y}$ is confined to the column space, this will be minimized at the closest point to Y in the subspace, meaning that $\hat{Y}$ is the orthogonal projection of Y onto the column span of X.
So $\hat{Y} = P_X Y$.

If I claim to you that

$$P_X = X(X^TX)^{-1}X^T$$

is the orthogonal projection operator onto the column span of X, then

$$\hat{Y} = P_X Y = X(X^TX)^{-1}X^TY = X\hat{\beta}$$

and therefore the minimizing parameters are given by

$$\hat{\beta} = (X^TX)^{-1}X^TY$$

We usually write $H = X(X^TX)^{-1}X^T$ and call this the hat matrix. Note that $\hat{Y} = HY$ is a linear transformation. You showed that H was symmetric and idempotent last Mini, which means that it is an orthogonal projection.

Regression: Traditional statistics vs. ML

Suppose that you observe your data X, Y, and fit a vector of coefficients $\hat{\beta}$. In traditional statistics you would do things like:

- Produce an interval $[L, U]$ that is likely to contain the next Y_i (prediction intervals)

- Determine how large β really is in the "true" model that generated the data. (Confidence intervals, hypothesis testing)

Regression: Traditional statistics vs. ML

To answer these questions, you need:

1. A model you believe for generating data, say:

$$Y_i = x_i^T \beta + \varepsilon_i$$

   where ε is mean zero and variance σ^2, and independent across observations. Or maybe $\varepsilon_i \sim N(0, \sigma^2)$
   This leads us to worry about diagnostics like residual plots, which might show that our linear model is wrong!

2. Distributions of $\hat{\beta}$ for given true β. Example:

$$\hat{\beta} \sim N(\beta, \sigma^2 (X^T X)^{-1})$$

   A variety of results of this sort exist for different model assumptions.

Regression: Traditional statistics vs. ML

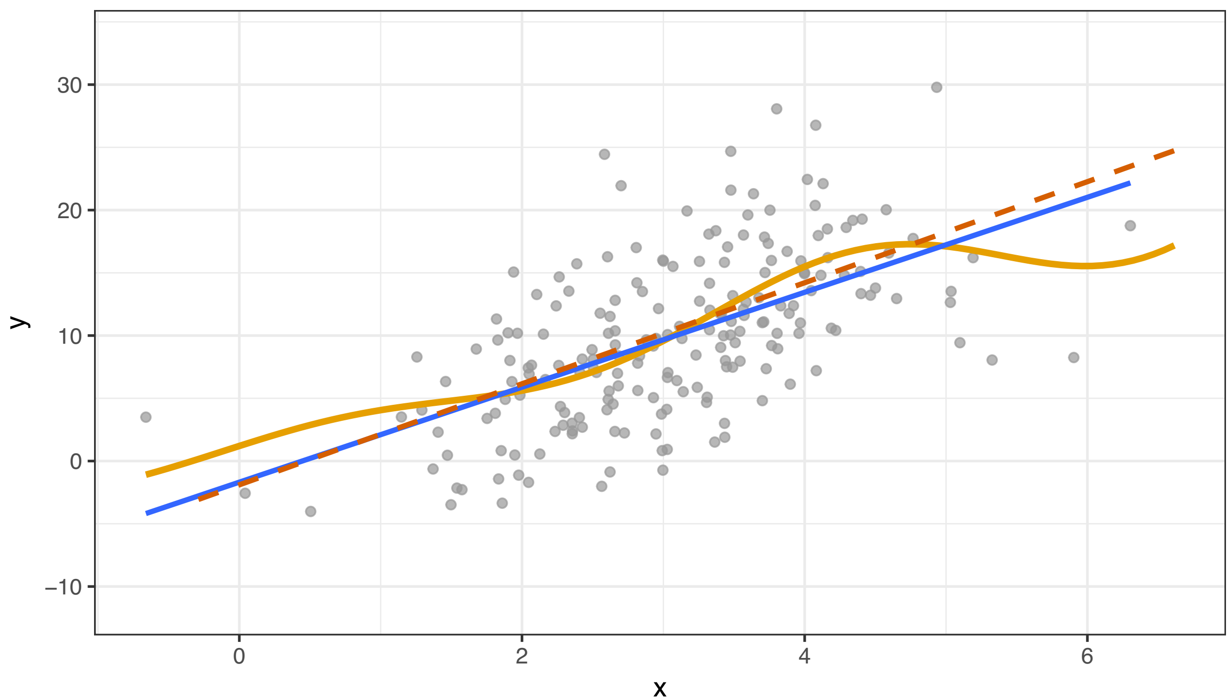
To get satisfying answers, you also worry about things like colinearity, meaning high correlations between predictors.

This is not really a problem for the math, but it tends to lead to difficulties in interpretation and in inference.

A prediction perspective

We care about all of the things above because we are trying to say something about the underlying “true” model.

From a prediction perspective, we might not really care about that true model!



Goals in modeling

In statistics and machine learning, we might fit a function $\hat{f}(x)$ to approximate a target $f(x)$. We could have several goals:

- Inference: Ask questions about $f(x)$.
- Interpretability: Ask questions about $\hat{f}(x)$
- Prediction: Guess Y using $\hat{f}(X)$.

Much of statistical inference focused on the first one. We will primarily care about the last two.

Goals in modeling

You have heard many horror stories about *colinearity* in regression. Suppose you fit a regression of Y on $X_1, X_2, \ldots$, but X_1 and X_2 are almost the same.

What happens to inference for linear regression?

What happens to predictions based on linear regression?

Linear regression as a predictor

Ok, so suppose that we use linear regression just to make predictions. What will happen?

You can obtain estimates $\hat{Y} = X\hat{\beta}$, and they might work well sometimes, even when the linear model is not true!

In particular, we know that least squares has nice properties when the linear model is true – e.g., it is unbiased and has minimum variance among unbiased, linear estimates (Gauss-Markov Theorem).

However...

Linear regression in high dimensions

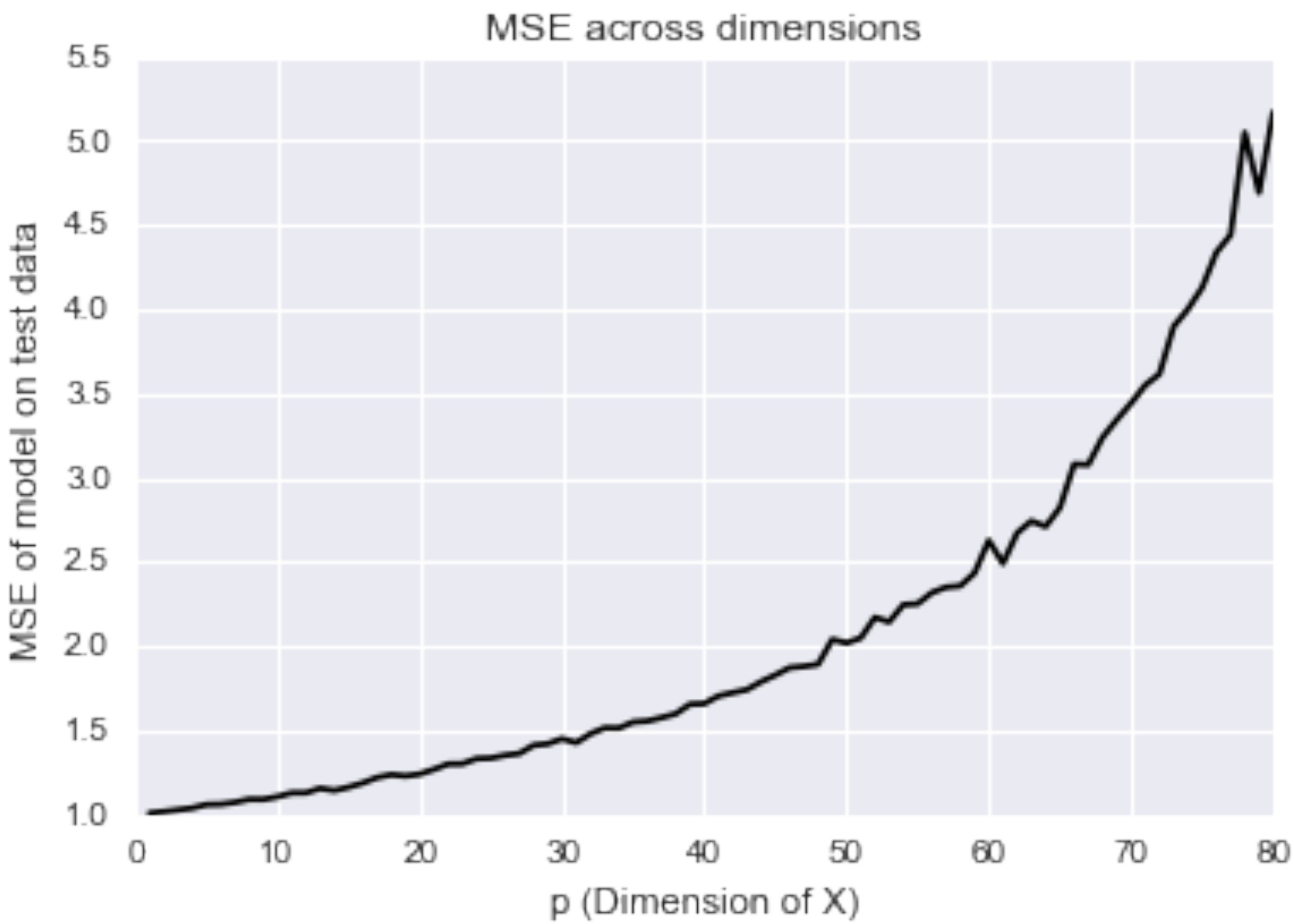
What happens when we have a bunch of variables? (Big data!)

We know that a linear model may perform terribly if the true function is far from linear.

Suppose that the linear model is even correct, so that $\mathbb{E}(Y|X)$ is exactly linear. What happens when we try to estimate it?

Linear regression in high dimensions

$E(y|x) = x\beta_p$



This is a simulation from a true linear model with independent Gaussian errors – the dream!

Linear regression in high dimensions

Suppose that the linear model is even correct, so that $\mathbb{E}(Y|X)$ is exactly linear. What happens when we try to estimate it?

As dimension increases, the prediction error increases (super-)linearly! When dimension is bigger than the number of observations,

What is going on, and how can we fix it?

Fixing linear regression in high dimensions

How do we reduce this error?

$$\mathbb{E}\left((Y - \hat{\mu}(X))^2\right) = \sigma_x^2 + \underbrace{\mathrm{Bias}\left(\hat{\mu}(X)\right)^2}_{} + \underbrace{\mathrm{Var}\left(\hat{\mu}(X)\right)}_{}$$